KRUTIK 🚨 CYBER EXPERT

═══
═══════

TERMUX + TELEGRAM BOT + PYTHON
ETHICAL HACKING COURSE

Termux Setup | Bot Automation | Security

BY KRUTIK 🚨
CYBER EXPERT

"Learn Free. Hack Ethical. Secure World."

📌 Educational Purpose Only

═══
═══════ **TERMAX ONLY FULL COURSE**
LINK 🔗 🖇️
https:You can access the full course document here: Termux Full Course.

═══
═══════
CHAPTER 1: WHAT IS TERMUX?
═══
═══════

What is Termux?
- Android Terminal Emulator + Linux Environment
- No Root Required
- Runs on Android 5.0+
- 600+ Linux Packages Available

Why Termux for Security Learning?
✅ Portable Penetration Testing Lab in your pocket
✅ Learn Linux commands anywhere
✅ Run Python scripts on mobile
✅ Practice networking and security concepts
✅ Free and open source

What Can You Do with Termux?
✓ Network Scanning (Nmap)
✓ Information Gathering (OSINT)
✓ Python Programming
✓ Telegram Bot Development
✓ Bash Scripting
✓ Cryptography Practice
✓ Learn Linux Commands

CHAPTER 2: TERMUX INSTALLATION & SETUP

A) INSTALLATION (Important - Use F-Droid)

⚠️ Google Play Store Version is Outdated!
Use F-Droid for latest stable version.

Step 1: Install F-Droid App
- Go to f-droid.org
- Download and install F-Droid APK

Step 2: Install Termux from F-Droid
- Open F-Droid
- Search "Termux"
- Install Termux (not Termux:Boot)

Step 3: Update Packages
Open Termux and run:

CODE:

```
pkg update && pkg upgrade -y
```

B) BASIC PACKAGES INSTALLATION

CODE:

```
# Essential packages
pkg install python git -y
pkg install python-pip -y
pkg install nano vim -y          # Text editors
```

```
# Security/Networking tools
pkg install nmap curl wget -y
pkg install dnsutils whois -y
pkg install net-tools -y

# Development tools
pkg install clang make -y
pkg install openssl -y
```

C) STORAGE ACCESS

CODE:

```
# Give Termux access to phone storage
termux-setup-storage

# After this, you can access files at:
# /storage/emulated/0/  (Internal Storage)

# Check if storage is mounted
ls /storage/emulated/0/
```

D) TERMUX BASIC COMMANDS

CODE:

```
# Navigate directories
ls          # List files
cd folder   # Change directory
pwd         # Show current path

# Create/Delete
mkdir folder   # Create folder
rm file.txt    # Delete file
rm -rf folder  # Delete folder

# File operations
cp file1 file2   # Copy
mv file1 file2   # Move/Rename
cat file.txt     # View file content

# Text editing
nano file.txt    # Open in nano editor
```

vim file.txt   # Open in vim editor

 # Permissions
chmod +x script.py # Make executable
chmod 777 file   # Full permissions

E) TERMUX KEYBOARD SHORTCUTS

Shortcuts:
- Volume Up + Q = Show/Hide Keyboard
- Volume Up + W = Ctrl (Send Ctrl+letter)
- Volume Up + C = Ctrl+C (Stop process)
- Volume Up + Z = Ctrl+Z (Background process)
- Volume Up + X = Ctrl+X (Exit)
- Volume Up + A = Ctrl+A (Move to line start)
- Volume Up + E = Ctrl+E (Move to line end)
- Volume Up + U = Ctrl+U (Delete line)
- Volume Up + K = Ctrl+K (Delete from cursor)
- Volume Up + L = Ctrl+L (Clear screen)

═══

CHAPTER 3: PYTHON ON TERMUX

═══

A) PYTHON INSTALLATION

CODE:

───

 # Install Python
pkg install python -y

 # Check version
python --version

 # Install pip (Python package manager)
pkg install python-pip -y

 # Or use pip directly
pip --version

 # Upgrade pip
pip install --upgrade pip

B) PYTHON PACKAGES FOR SECURITY

CODE:

```
# Core packages
pip install requests
pip install beautifulsoup4
pip install scapy

# Telegram Bot packages
pip install pyTelegramBotAPI
pip install telethon
pip install python-telegram-bot

# Networking
pip install socket
pip install paramiko  # SSH

# Cryptography
pip install cryptography
pip install pycryptodome

# API & Automation
pip install selenium
pip install pyautogui
```

C) RUNNING PYTHON SCRIPTS

CODE:

```
# Create a Python file
nano first.py

# Write code in the file
print("Hello from Termux!")
print("Python works on mobile!")

# Save and exit (Ctrl+O, Enter, Ctrl+X)

# Run the file
python first.py

# Or with arguments
python script.py --help
```

D) PYTHON VIRTUAL ENVIRONMENT

CODE:

```
# Install venv
pkg install python-venv

# Create virtual environment
python -m venv myenv

# Activate virtual environment
source myenv/bin/activate

# Deactivate
deactivate
```

CHAPTER 4: TELEGRAM BOT WITH PYTHON

A) CREATING A TELEGRAM BOT

Step 1: Open Telegram and search @BotFather
Step 2: Send /newbot
Step 3: Choose a name for your bot (e.g., MySecurityBot)
Step 4: Choose a username (must end with 'bot')
Step 5: Copy the API Token (e.g., 123456:ABC-DEF)

B) PYTHON BOT - BASIC SETUP

CODE:

```
# Install the library
pip install pyTelegramBotAPI

# Create bot.py
nano bot.py
```

CODE (bot.py):

```
import telebot

# Replace with your bot token
BOT_TOKEN = "YOUR_BOT_TOKEN_HERE"

# Initialize bot
bot = telebot.TeleBot(BOT_TOKEN)

# Handler for /start and /hello commands
@bot.message_handler(commands=['start', 'hello'])
def send_welcome(message):
    bot.reply_to(message, f"Hello {message.from_user.first_name}! 🚀\nI am your
Security Assistant Bot.")

# Handler for /help command
@bot.message_handler(commands=['help'])
def send_help(message):
    help_text = """
    🤖 Available Commands:
    /start - Welcome message
    /help - Show this help
    /info - Get your info
    /echo <text> - Echo your message
    /scan <ip> - Basic port scan (Educational)
    /ping <ip> - Ping an IP address
    """
    bot.reply_to(message, help_text)

# Handler for /info command
@bot.message_handler(commands=['info'])
def send_info(message):
    user = message.from_user
    info_text = f"""
    📋 Your Information:
    ID: {user.id}
    Name: {user.first_name} {user.last_name or ''}
    Username: @{user.username or 'Not set'}
    Language: {user.language_code or 'Unknown'}
    """
    bot.reply_to(message, info_text)

# Handler for /echo command
@bot.message_handler(commands=['echo'])
def echo_command(message):
    text = message.text.replace('/echo', '').strip()
    if text:
        bot.reply_to(message, f"Echo: {text}")
    else:
```

```
    bot.reply_to(message, "Please provide text to echo. Example: /echo Hello
World!")

# Handler for /ping command (Educational)
@bot.message_handler(commands=['ping'])
def ping_command(message):
    import subprocess
    import re

    # Get IP from command
    parts = message.text.split()
    if len(parts) < 2:
        bot.reply_to(message, "Please provide an IP. Example: /ping 8.8.8.8")
        return

    ip = parts[1]

    # Validate IP format
    if not re.match(r'^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$', ip):
        bot.reply_to(message, "Invalid IP format. Use: 8.8.8.8")
        return

    # Send "processing" message
    bot.reply_to(message, f"⏳ Pinging {ip}... (Educational purpose only)")

    # Note: Ping in Termux requires root, this is for demonstration
    # In real educational scenarios, you'd use socket or other methods
    bot.reply_to(message, f"✅ Ping to {ip} simulated successfully!")

# Handler for all text messages
@bot.message_handler(func=lambda msg: True)
def echo_all(message):
    bot.reply_to(message, f"You said: {message.text}\n\nType /help for available
commands.")

# Start the bot
print("🤖 Bot is running...")
print("Press Ctrl+C to stop")
bot.infinity_polling()
```

C) RUNNING THE BOT

CODE:

```
# Run in background (with nohup)
nohup python bot.py &
```

```
# Or use tmux/screen for persistent session
pkg install tmux
tmux new -s bot
python bot.py
# Detach: Ctrl+B, D
# Reattach: tmux attach -t bot

# Kill the bot
pkill -f bot.py
```

D) ADVANCED BOT FEATURES

CODE (Advanced Features):

```
import telebot
import subprocess
import os
import time
import requests
import json

BOT_TOKEN = "YOUR_BOT_TOKEN_HERE"
ADMIN_ID = YOUR_TELEGRAM_ID  # Replace with your ID

bot = telebot.TeleBot(BOT_TOKEN)

# Admin only command
@bot.message_handler(commands=['admin'])
def admin_command(message):
    if message.from_user.id != ADMIN_ID:
        bot.reply_to(message, "⛔ Unauthorized access!")
        return

    bot.reply_to(message, "✅ Welcome Admin!")

# Get system info
@bot.message_handler(commands=['sysinfo'])
def sysinfo_command(message):
    if message.from_user.id != ADMIN_ID:
        bot.reply_to(message, "⛔ Admin only command!")
        return

    info = f"""
    📱 System Information:
    Hostname: {os.uname().nodename}
```

```
    OS: {os.uname().sysname}
    Release: {os.uname().release}
    Version: {os.uname().version}
    Python: {subprocess.getoutput('python --version')}
    """
    bot.reply_to(message, info)

# Make HTTP request (Educational)
@bot.message_handler(commands=['get'])
def get_command(message):
    parts = message.text.split()
    if len(parts) < 2:
        bot.reply_to(message, "Usage: /get <url>")
        return

    url = parts[1]
    try:
        response = requests.get(url, timeout=10)
        content = response.text[:1000]  # Limit response
        bot.reply_to(message, f"Response from {url}:\n{content}")
    except Exception as e:
        bot.reply_to(message, f"Error: {str(e)}")

# Inline keyboard example
@bot.message_handler(commands=['menu'])
def menu_command(message):
    markup = telebot.types.InlineKeyboardMarkup()
    markup.row(
        telebot.types.InlineKeyboardButton("🔍 Scan", callback_data="scan"),
        telebot.types.InlineKeyboardButton("📊 Info", callback_data="info")
    )
    markup.row(
        telebot.types.InlineKeyboardButton("💬 Help", callback_data="help"),
        telebot.types.InlineKeyboardButton("❌ Close", callback_data="close")
    )
    bot.reply_to(message, "📋 Select an option:", reply_markup=markup)

@bot.callback_query_handler(func=lambda call: True)
def callback_query(call):
    if call.data == "scan":
        bot.answer_callback_query(call.id, "🔍 Scanning...")
        bot.reply_to(call.message, "⚠️ Scan feature for educational use only!")
    elif call.data == "info":
        bot.answer_callback_query(call.id, "📊 Info")
        bot.reply_to(call.message, f"Your ID: {call.from_user.id}")
    elif call.data == "help":
        bot.answer_callback_query(call.id, "💬 Help")
        bot.reply_to(call.message, "Type /help for available commands")
```

```
    elif call.data == "close":
        bot.answer_callback_query(call.id, "❌ Closed")
        bot.delete_message(call.message.chat.id, call.message.message_id)
```

==========

CHAPTER 5: ETHICAL HACKING TOOLS ON TERMUX

==========

A) NMAP - NETWORK SCANNER

CODE:

```
# Install Nmap
pkg install nmap -y

# Basic scan
nmap scanme.nmap.org

# Scan specific port
nmap -p 80,443 scanme.nmap.org

# Scan range of ports
nmap -p 1-1000 scanme.nmap.org

# Version detection
nmap -sV scanme.nmap.org

# OS detection
nmap -O scanme.nmap.org

# Aggressive scan
nmap -A scanme.nmap.org

# Save output
nmap -oN scan_result.txt scanme.nmap.org
```

B) PYTHON PORT SCANNER (Educational)

CODE (port_scanner.py):

```
#!/usr/bin/env python3
"""
```

```
Educational Port Scanner
Only use on systems you own or have permission to test
"""

import socket
import sys
import threading
from datetime import datetime

def scan_port(ip, port):
    """Scan a single port"""
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(0.5)
        result = sock.connect_ex((ip, port))
        if result == 0:
            return port, True
        sock.close()
        return port, False
    except:
        return port, False

def scan_ports(ip, start_port=1, end_port=1024):
    """Scan a range of ports"""
    print(f"\n🔍 Scanning {ip}...")
    print(f"Ports: {start_port}-{end_port}")
    print("=" * 50)

    open_ports = []
    threads = []

    for port in range(start_port, end_port + 1):
        t = threading.Thread(target=lambda: open_ports.append(port) if scan_port(ip,
port)[1] else None)
        threads.append(t)
        t.start()

    # Wait for all threads
    for t in threads:
        t.join()

    return sorted(open_ports)

def main():
    print("=" * 50)
    print("🔒 EDUCATIONAL PORT SCANNER")
    print("Only use on systems you own!")
    print("=" * 50)
```

```
    # Get target
    target = input("Enter target IP: ").strip()

    # Validate IP
    parts = target.split('.')
    if len(parts) != 4:
        print("❌ Invalid IP format!")
        return

    for part in parts:
        if not part.isdigit():
            print("❌ Invalid IP format!")
            return

    try:
        start = int(input("Start port (default 1): ") or 1)
        end = int(input("End port (default 1024): ") or 1024)
    except:
        start, end = 1, 1024

    print(f"\n⏳ Starting scan at {datetime.now()}")

    open_ports = scan_ports(target, start, end)

    print(f"\n✅ Scan completed at {datetime.now()}")
    print("=" * 50)

    if open_ports:
        print(f"🔓 Open ports found: {len(open_ports)}")
        for port in open_ports:
            print(f"  Port {port}: OPEN")
    else:
        print("🔒 No open ports found in this range")

    print("=" * 50)

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n\n❌ Scan interrupted by user")
        sys.exit()
```

C) INFORMATION GATHERING (OSINT) - Python Script

CODE (osint_info.py):

```
#!/usr/bin/env python3
"""
Educational OSINT Tool
Gathers publicly available information
"""

import socket
import requests
import whois
import dns.resolver

def get_ip_info(ip):
    """Get basic IP information"""
    info = {}

    try:
        # Get hostname
        info['hostname'] = socket.gethostbyaddr(ip)[0]
    except:
        info['hostname'] = "Not found"

    try:
        # Get geolocation (free API)
        response = requests.get(f"http://ip-api.com/json/{ip}", timeout=5)
        data = response.json()
        if data['status'] == 'success':
            info['country'] = data['country']
            info['city'] = data['city']
            info['isp'] = data['isp']
            info['org'] = data['org']
    except:
        info['geo'] = "Not available"

    return info

def get_domain_info(domain):
    """Get domain information"""
    info = {}

    try:
        # WHOIS lookup
        w = whois.whois(domain)
        info['registrar'] = w.registrar
        info['creation_date'] = w.creation_date
        info['expiration_date'] = w.expiration_date
```

```
            info['name_servers'] = w.name_servers
        except:
            info['whois'] = "Not available"

        try:
            # DNS lookup
            info['ips'] = []
            for record in dns.resolver.resolve(domain, 'A'):
                info['ips'].append(str(record))
        except:
            info['dns'] = "Not available"

        return info

    def main():
        print("=" * 50)
        print("🔍 EDUCATIONAL OSINT TOOL")
        print("Only for educational purposes!")
        print("=" * 50)

        print("\n1. IP Lookup")
        print("2. Domain Lookup")
        print("3. Exit")

        choice = input("\nSelect option: ").strip()

        if choice == "1":
            ip = input("Enter IP address: ").strip()
            info = get_ip_info(ip)
            print("\n📋 IP Information:")
            for key, value in info.items():
                print(f"  {key}: {value}")

        elif choice == "2":
            domain = input("Enter domain (example.com): ").strip()
            info = get_domain_info(domain)
            print("\n📋 Domain Information:")
            for key, value in info.items():
                print(f"  {key}: {value}")

        else:
            print("Goodbye!")

    if __name__ == "__main__":
        try:
            main()
        except KeyboardInterrupt:
            print("\n\nExited.")
```

```
    except Exception as e:
        print(f"Error: {e}")
```

D) CYBERTUZ - ALL-IN-ONE LEARNING PLATFORM

CODE:

```
# Install Cybertuz
pip install cypertuz

# Run Cybertuz
cybertuz

# Or clone and run
git clone https://github.com/darkfanx/cypertuz
cd cypertuz
python cybertuz.py
```

E) OTHER USEFUL TOOLS

CODE:

```
# Hydra - Brute force attack (educational only)
pkg install hydra -y
hydra -l admin -P passlist.txt ssh://192.168.1.1

# John the Ripper - Password cracking (educational only)
pkg install john -y

# SQLmap - SQL Injection (educational only)
git clone https://github.com/sqlmapproject/sqlmap
cd sqlmap
python sqlmap.py --help

# Metasploit - Advanced Penetration Testing (Educational only)
# Note: May need additional setup
git clone https://github.com/rapid7/metasploit-framework
cd metasploit-framework
gem install bundler
bundle install
```

CHAPTER 6: TELEGRAM BOT WITH TELEGRAM CLIENT (Telethon)

A) INSTALL TELEGRAM AND GENERATE SESSION STRING

CODE:

```
# Install Telethon
pip install telethon

# Create session_generator.py
nano session_generator.py
```

CODE (session_generator.py):

```
from telethon import TelegramClient
from telethon.sessions import StringSession

print("=" * 50)
print("🔑 TELEGRAM SESSION GENERATOR")
print("Educational Purpose Only")
print("=" * 50)

# Get API credentials from my.telegram.org
api_id = int(input("\nEnter API ID: ").strip())
api_hash = input("Enter API Hash: ").strip()

# Generate session
with TelegramClient(StringSession(), api_id, api_hash) as client:
    print("\n✅ Login successful!")
    session_string = client.session.save()
    print("\n📋 Your Session String:")
    print(session_string)
    print("\n⚠️ Keep this secret! Never share with anyone!")
```

B) BASIC TELEGRAM CLIENT BOT

CODE (telegram_client.py):

```
from telethon import TelegramClient, events
import asyncio
```

```
# Replace with your values
API_ID = YOUR_API_ID
API_HASH = "YOUR_API_HASH"
SESSION_STRING = "YOUR_SESSION_STRING"

# Initialize client
client = TelegramClient(StringSession(SESSION_STRING), API_ID, API_HASH)

@client.on(events.NewMessage(incoming=True))
async def handle_message(event):
    """Handle incoming messages"""
    if event.is_private:
        print(f"Message from {event.sender_id}: {event.message.text}")
        if event.message.text.lower() == "ping":
            await event.reply("Pong!")
        elif event.message.text.lower() == "help":
            await event.reply("Commands: ping, help, time")

@client.on(events.NewMessage(outgoing=True))
async def handle_outgoing(event):
    """Handle outgoing messages"""
    print(f"Sent: {event.message.text}")

async def main():
    print("🤖 Telegram Client Started")
    print("Listening for messages...")
    await client.start()
    await client.run_until_disconnected()

if __name__ == "__main__":
    asyncio.run(main())
```

C) MESSAGE MONITORING BOT (Educational)

CODE (monitor_bot.py):

```
from telethon import TelegramClient, events
import asyncio

API_ID = YOUR_API_ID
API_HASH = "YOUR_API_HASH"
SESSION_STRING = "YOUR_SESSION_STRING"

# Channel/Group to monitor
MONITOR_CHANNEL = "@channel_username"
```

```
client = TelegramClient(StringSession(SESSION_STRING), API_ID, API_HASH)

@client.on(events.NewMessage(chats=MONITOR_CHANNEL))
async def monitor_messages(event):
    """Monitor messages in a channel"""
    message = event.message.text
    sender = await event.get_sender()

    print(f"\n📩 New message from {MONITOR_CHANNEL}")
    print(f"Sender: {sender.first_name}")
    print(f"Message: {message[:100]}...")

async def main():
    print(f"🔍 Monitoring {MONITOR_CHANNEL}...")
    await client.start()
    await client.run_until_disconnected()

if __name__ == "__main__":
    asyncio.run(main())
```

CHAPTER 7: COMPLETE PROJECT - SECURITY BOT

CODE (security_bot.py):

```
#!/usr/bin/env python3
"""
Telegram Security Bot - Educational Purpose
Features:
- /start, /help, /info
- /ping <ip> - Ping simulation
- /scan <ip> - Port scan (educational)
- /whois <domain> - WHOIS lookup
- /dns <domain> - DNS lookup
- /geo <ip> - Geolocation
- /date - Current date/time
- /stats - Bot statistics
"""

import telebot
import socket
import requests
import datetime
```

```
import threading
import json
import time
import re
import whois

# Configuration
BOT_TOKEN = "YOUR_BOT_TOKEN_HERE"

# Initialize bot
bot = telebot.TeleBot(BOT_TOKEN)

# Statistics
stats = {
    'users': set(),
    'total_commands': 0,
    'start_time': time.time()
}

# ============ COMMAND HANDLERS ============

@bot.message_handler(commands=['start', 'help'])
def welcome(message):
    help_text = """
    🤖 **Security Bot** - Educational Purpose Only

    📌 **Available Commands:**

    🔹 /start - Welcome message
    🔹 /help - Show this help
    🔹 /info - Your user info
    🔹 /ping <ip> - Ping an IP
    🔹 /scan <ip> - Port scan (1-1024)
    🔹 /whois <domain> - WHOIS lookup
    🔹 /dns <domain> - DNS lookup
    🔹 /geo <ip> - Geolocation
    🔹 /date - Current date/time
    🔹 /stats - Bot statistics

    ⚠️ **Disclaimer:** Educational use only!
    """
    bot.reply_to(message, help_text, parse_mode='Markdown')

@bot.message_handler(commands=['info'])
def user_info(message):
    user = message.from_user
    info_text = f"""
    📋 **User Information**
```

```
    ID: `{user.id}`
    Name: {user.first_name} {user.last_name or ''}
    Username: @{user.username or 'Not set'}
    Language: {user.language_code or 'Unknown'}

    Chat Type: {message.chat.type}
    """
    bot.reply_to(message, info_text, parse_mode='Markdown')
    update_stats(message)

@bot.message_handler(commands=['ping'])
def ping_command(message):
    parts = message.text.split()
    if len(parts) < 2:
        bot.reply_to(message, "❌ Please provide an IP.\nExample: `/ping 8.8.8.8`",
parse_mode='Markdown')
        return

    ip = parts[1]

    # Validate IP
    if not re.match(r'^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$', ip):
        bot.reply_to(message, "❌ Invalid IP format. Use: 8.8.8.8")
        return

    # Simulate ping
    bot.reply_to(message, f"⏳ Pinging {ip}...")

    # For educational purposes, we'll do a simple check
    try:
        import subprocess
        result = subprocess.run(['ping', '-c', '1', '-W', '2', ip],
                                capture_output=True, text=True, timeout=5)
        if result.returncode == 0:
            response = f"✅ Ping to {ip} successful!\n\n{result.stdout[:500]}"
        else:
            response = f"❌ Ping to {ip} failed.\n\n{result.stderr[:500]}"
    except Exception as e:
        response = f"⚠️ Ping simulation: {ip} is reachable! (Educational mode)"

    bot.reply_to(message, response[:4000])
    update_stats(message)

@bot.message_handler(commands=['scan'])
def scan_command(message):
    parts = message.text.split()
    if len(parts) < 2:
```

```
        bot.reply_to(message, "❌ Please provide an IP.\nExample: `/scan 192.168.1.1`",
parse_mode='Markdown')
        return

    ip = parts[1]

    # Validate IP
    parts_ip = ip.split('.')
    if len(parts_ip) != 4 or not all(p.isdigit() for p in parts_ip):
        bot.reply_to(message, "❌ Invalid IP format")
        return

    # Ask for port range
    bot.reply_to(message, f"🔍 Scanning {ip}... (Common ports 1-1024)\n⏳ This may
take a few moments...")

    # Run scan in thread
    def scan_thread():
        open_ports = []

        for port in range(1, 1025):
            try:
                sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
                sock.settimeout(0.3)
                result = sock.connect_ex((ip, port))
                if result == 0:
                    open_ports.append(port)
                sock.close()
            except:
                pass

        if open_ports:
            response = f"✅ Scan complete for {ip}\n\n🔓 **Open Ports:**\n"
            for port in open_ports:
                service = get_service_name(port)
                response += f"  Port {port}: {service}\n"
        else:
            response = f"✅ Scan complete for {ip}\n\n🔒 No open ports found in range
1-1024"

        bot.send_message(message.chat.id, response[:4000], parse_mode='Markdown')
        update_stats(message)

    threading.Thread(target=scan_thread).start()

def get_service_name(port):
    services = {
        21: "FTP", 22: "SSH", 23: "Telnet", 25: "SMTP",
```

```
        53: "DNS", 80: "HTTP", 110: "POP3", 143: "IMAP",
        443: "HTTPS", 3306: "MySQL", 3389: "RDP",
        5432: "PostgreSQL", 6379: "Redis", 8080: "HTTP-Alt"
    }
    return services.get(port, "Unknown")

@bot.message_handler(commands=['whois'])
def whois_command(message):
    parts = message.text.split()
    if len(parts) < 2:
        bot.reply_to(message, "❌ Please provide a domain.\nExample: `/whois
example.com`", parse_mode='Markdown')
        return

    domain = parts[1]

    # Validate domain
    if not re.match(r'^[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$', domain):
        bot.reply_to(message, "❌ Invalid domain format")
        return

    bot.reply_to(message, f"⏳ Looking up {domain}...")

    try:
        w = whois.whois(domain)
        response = f"""
📋 **WHOIS Information for {domain}**

**Registrar:** {w.registrar or 'N/A'}
**Creation Date:** {w.creation_date or 'N/A'}
**Expiration Date:** {w.expiration_date or 'N/A'}
**Name Servers:** {', '.join(w.name_servers) if w.name_servers else 'N/A'}
**Status:** {', '.join(w.status) if w.status else 'N/A'}
"""
        bot.reply_to(message, response[:4000], parse_mode='Markdown')
    except Exception as e:
        bot.reply_to(message, f"❌ Error: Could not lookup {domain}\n\n{str(e)}")

    update_stats(message)

@bot.message_handler(commands=['dns'])
def dns_command(message):
    parts = message.text.split()
    if len(parts) < 2:
        bot.reply_to(message, "❌ Please provide a domain.\nExample: `/dns
example.com`", parse_mode='Markdown')
        return
```

```
    domain = parts[1]

    bot.reply_to(message, f"⏳ Looking up DNS for {domain}...")

    try:
        import dns.resolver
        response = f"📋 **DNS Records for {domain}**\n\n"

        records = {
            'A': 'IPv4 Addresses',
            'AAAA': 'IPv6 Addresses',
            'MX': 'Mail Exchangers',
            'NS': 'Name Servers',
            'TXT': 'TXT Records',
            'CNAME': 'Canonical Name'
        }

        for record_type, description in records.items():
            try:
                answers = dns.resolver.resolve(domain, record_type)
                response += f"\n**{description} ({record_type}):**\n"
                for r in answers:
                    response += f"  {r}\n"
            except:
                pass

        if response == f"📋 **DNS Records for {domain}**\n\n":
            response += "No DNS records found."

        bot.reply_to(message, response[:4000], parse_mode='Markdown')
    except Exception as e:
        bot.reply_to(message, f"❌ Error: {str(e)}")

    update_stats(message)

@bot.message_handler(commands=['geo'])
def geo_command(message):
    parts = message.text.split()
    if len(parts) < 2:
        bot.reply_to(message, "❌ Please provide an IP.\nExample: `/geo 8.8.8.8`",
parse_mode='Markdown')
        return

    ip = parts[1]

    bot.reply_to(message, f"⏳ Looking up location for {ip}...")

    try:
```

```
        response = requests.get(f"http://ip-api.com/json/{ip}", timeout=5)
        data = response.json()

        if data['status'] == 'success':
            response_text = f"""
📋 **Geolocation for {ip}**

**Country:** {data.get('country', 'N/A')}
**Region:** {data.get('regionName', 'N/A')}
**City:** {data.get('city', 'N/A')}
**ISP:** {data.get('isp', 'N/A')}
**Organization:** {data.get('org', 'N/A')}
**Latitude:** {data.get('lat', 'N/A')}
**Longitude:** {data.get('lon', 'N/A')}
**Timezone:** {data.get('timezone', 'N/A')}
"""
        else:
            response_text = f"❌ Could not get location for {ip}"

        bot.reply_to(message, response_text[:4000], parse_mode='Markdown')
    except Exception as e:
        bot.reply_to(message, f"❌ Error: {str(e)}")

    update_stats(message)

@bot.message_handler(commands=['date'])
def date_command(message):
    now = datetime.datetime.now()
    response = f"""
📅 **Current Date/Time**

**Date:** {now.strftime('%B %d, %Y')}
**Time:** {now.strftime('%H:%M:%S')}
**Day:** {now.strftime('%A')}
**Week:** {now.isocalendar()[1]}
**Timezone:** {datetime.datetime.now().astimezone().tzinfo}
"""
    bot.reply_to(message, response, parse_mode='Markdown')
    update_stats(message)

@bot.message_handler(commands=['stats'])
def stats_command(message):
    uptime = time.time() - stats['start_time']
    hours = int(uptime // 3600)
    minutes = int((uptime % 3600) // 60)

    response = f"""
📊 **Bot Statistics**
```

```
**Users:** {len(stats['users'])}
**Commands Processed:** {stats['total_commands']}
**Uptime:** {hours}h {minutes}m
**Started:** {datetime.datetime.fromtimestamp(stats['start_time']).strftime('%Y-%m-%d
%H:%M:%S')}
"""
    bot.reply_to(message, response, parse_mode='Markdown')

# ============ UTILITY FUNCTIONS ============

def update_stats(message):
    """Update bot statistics"""
    stats['users'].add(message.from_user.id)
    stats['total_commands'] += 1

# ============ MAIN ============

def main():
    print("=" * 50)
    print("🤖 SECURITY BOT")
    print("Educational Purpose Only")
    print("=" * 50)
    print(f"Bot Token: {BOT_TOKEN[:10]}...")
    print("Starting bot...")

    try:
        print("\n✅ Bot is running!")
        print("Press Ctrl+C to stop")
        print("=" * 50)
        bot.infinity_polling()
    except Exception as e:
        print(f"❌ Error: {e}")

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n\n🔲 Bot stopped by user")
```

CHAPTER 8: SECURITY & ETHICS

A) CYBER LAWS (INDIA - IT ACT 2000)

Section	Penalty
Sec 43	Fine up to ₹1 Lakh
Sec 66	3 Years Jail + ₹5 Lakh
Sec 66B	3 Years Jail + ₹1 Lakh
Sec 66C	3 Years Jail + ₹1 Lakh
Sec 66D	3 Years Jail + ₹1 Lakh
Sec 67	3 Years Jail + ₹2 Lakh
Sec 70	10 Years Jail + ₹1 Crore

B) RED FLAGS (MALWARE IN TERMUX)

1. ⚠️ Never run unknown scripts without reviewing
2. ⚠️ Don't share your session strings
3. ⚠️ Avoid downloading APK from untrusted sources
4. ⚠️ Don't give Termux root access unless necessary
5. ⚠️ Be careful with permissions (storage, contacts)

C) GOLDEN RULES OF ETHICAL HACKING

✅ Rule 1: Always get written permission
✅ Rule 2: Only test systems you own
✅ Rule 3: Report vulnerabilities responsibly
✅ Rule 4: Never cause harm or damage
✅ Rule 5: Respect privacy and data
✅ Rule 6: Keep learning and stay updated
✅ Rule 7: Never use skills for illegal activities

D) SAFETY TIPS FOR TERMUX

CODE:

```
# Check what's running
ps aux

# Check network connections
netstat -tulpn

# Check recent commands
history
```

 # Check file permissions
ls -la

 # Monitor system resources
top

═══

CHAPTER 9: QUICK REFERENCE

═══

COMMON TERMUX COMMANDS:
pkg update && pkg upgrade -y
pkg install <package>
python script.py
ls, cd, pwd, mkdir, rm, mv, cp

PYTHON PACKAGES:
pip install <package>
pip list
pip uninstall <package>

TELEGRAM BOT COMMANDS:
/start - Start bot
/help - Help menu
/info - User info
/ping <ip> - Ping IP
/scan <ip> - Port scan
/whois <domain> - WHOIS lookup
/dns <domain> - DNS lookup
/geo <ip> - Geolocation
/date - Current date/time
/stats - Bot statistics

SECURITY TOOLS:
nmap - Network scanner
hydra - Brute force
john - Password cracking
sqlmap - SQL injection
metasploit - Penetration testing

═══

CERTIFICATE OF COMPLETION

═══

This certifies that the reader has completed

TERMUX + TELEGRAM BOT + PYTHON
ETHICAL HACKING COURSE

By
KRUTIK 🚨 CYBER EXPERT

Topics Covered:
✓ Termux Installation & Setup (F-Droid)
✓ Essential Linux Commands
✓ Python on Termux (Packages, Virtual Environment)
✓ Telegram Bot Creation (BotFather)
✓ Python Telegram Bot (pyTelegramBotAPI)
✓ Telegram Client (Telethon - Session String)
✓ Network Scanning (Nmap, Python Port Scanner)
✓ Information Gathering (OSINT Tools)
✓ DNS & WHOIS Lookup
✓ Geolocation APIs
✓ Security Bot Project (Complete)
✓ Cyber Laws (IT Act 2000)
✓ Safety & Ethics

"Knowledge is Power. Use it Wisely."

⚠️ DISCLAIMER: This course is strictly for EDUCATIONAL PURPOSE ONLY.
All techniques demonstrated are for learning cybersecurity concepts.
The creator (Krutik 🚨 Cyber Expert) does not encourage or support
any illegal activities. Users are solely responsible for their actions.

"Always get permission before testing any system."
